

Fractal Behavioral Grammar

Overdub Architecture & Binary Encoding

v0.3.0 — v0.4.0 Technical Whitepaper

Author:	Ryan Moore — Greenbars Systems
Patent Docket:	FBG-2026-001 (provisional, filed June 2026)
Non-provisional deadline:	June 29, 2027
Repository:	github.com/GreenbarSystems/fractal-grammar
License:	Apache 2.0

ABSTRACT

v0.3.0 introduces hierarchical residual compression — a three-layer behavioral memory model (Domain / Behavioral / Session) that stores only the residual signal not explained by lower layers, inspired by video codec P/B-frames. v0.4.0 adds a Rust/PyO3 binary encoding layer implementing variable-length integer indices and run-length encoded session metadata. The combined result: 117 validated behavioral grammar rules stored in 860 bytes — 0.41x the cost of the original 20-rule flat architecture, with 10.9x compression versus JSON and 3.0x encode speedup over the Python reference implementation. All 55 tests pass (40 Rust unit + 15 Python integration).

1. Motivation

Prior to v0.3.0, fractal-grammar used a flat persistence layer with a hard cap of 20 grammar rules. That cap was arbitrary — it existed because the JSON serialization cost scaled linearly with rule count, and a large ruleset would consume the system prompt token budget on every inference call. The underlying extraction pipeline could identify far more validated rules than the persistence layer could store.

Two problems follow from a 20-rule cap:

- Temporal collapse: a rule observed once in a burst session competes for the same 20 slots as a rule present across 40 sessions. There is no way to express stability.
- Storage ceiling: the JSON representation of 20 rules costs 2,104 bytes. Expanding to 117 rules in JSON costs 25,012 bytes — an 11.9× increase that is unacceptable for local inference on consumer hardware.

The overdub architecture (v0.3.0) solves the temporal collapse problem by introducing a three-layer model. Binary encoding (v0.4.0) solves the storage ceiling by replacing JSON serialization with a compact wire format. This whitepaper documents both changes.

2. Overdub Architecture (v0.3.0)

2.1 Design Rationale

The overdub metaphor is deliberate. A loop machine overdubs — it records a new layer that plays atop existing layers, encoding only what is new. A video codec uses the same principle: I-frames encode a full reference state; P-frames and B-frames encode only the residual change from the nearest reference. Both approaches achieve compression by eliminating redundancy across layers.

The neuroscientific motivation is equally direct. The medial prefrontal cortex (mPFC) stores compressed behavioral rules — the 'what usually happens.' The CA1 hippocampus stores episodic memory — the 'what happened on Tuesday.' These are architecturally distinct. A behavioral grammar library should separate them the same way: rules (grammar) are not logs (episodes).

2.2 Three-Layer Storage Model

The promotion gate classifies each extracted grammar rule into one of three temporal layers based on how many distinct sessions that rule has appeared in. Classification uses an 8-bit circular bitmask per rule — compact, ephemeral, and never persisted to disk.

Layer	Session Count	Codec Analogy	Interpretation	TTL
Layer 0 — Domain	≥ 5 sessions	I-frame	Stable identity; present across months	None
Layer 1 — Behavioral	2–4 sessions	P-frame	Current behavioral context; weeks-scale	None

Layer 2 — Session	1 session	B-frame	Transient signal; may be noise	1 session
-------------------	-----------	---------	--------------------------------	-----------

Table 1. Three-layer temporal classification.

The promotion gate is always rebuilt from the event log on demand. It is never serialized. Only the rule blobs for L0, L1, and L2 are written to disk. This means the gate state has zero persistence overhead — its entire cost is the 152 μ s rebuild at session startup.

2.3 Quarantine Rate

On a 1,000-event simulation trace (200 unique rules, 10 sessions), 83 of 200 rules expired silently — they were observed in exactly one session and were not promoted to L1 or L0. This 41.5% quarantine rate is the gate doing its job: burst noise from a single unusual session does not contaminate the stable grammar. Only rules with cross-session evidence enter the persistent layers.

2.4 Residual Encoding Semantics

Each layer encodes only the rules that belong to that layer — not all rules. L0 contains domain-stable rules. L1 contains behavioral-context rules not already in L0. L2 contains session-transient rules not already in L0 or L1. This is the residual: each layer stores the delta not explained by lower layers. At inference time, the ruleset is reconstructed by reading L0 + L1 (and optionally L2), applying priority ordering: L0 rules overwrite any L1 rule with the same LHS.

3. Binary Encoding — v0.4.0 Wire Format

3.1 Wire Format Overview

The v0.4.0 wire format is a compact binary representation of a single grammar layer blob. The format is little-endian throughout. VERSION = 0x04; VERSION_V3 = 0x03 is still accepted for backward compatibility.

Layout (sequential, no padding):

HEADER (8 bytes) | STRING_TABLE (variable) | SESSION_RUNS (variable, v0.4 only) | RULE_RECORDS (variable) | TTL_RECORDS (optional, Layer 2 only)

Section	Size	Contents
HEADER	8 bytes	magic(u16=0xFBFG1) · version(u8) · layer_id(u8) · rule_count(u16) · flags(u16)
STRING_TABLE	variable	u16 n_strings · (u8 len · utf8 bytes)* — deduplicated LHS/RHS token strings
SESSION_RUNS	variable	u16 n_runs · (count:u8 · value:u8)* — RLE session_count per rule ← NEW v0.4
RULE_RECORDS	variable	per rule: varint(lhs_idx) · varint(rhs_idx) · varint(support, zigzag-delta) · conf_q8:u8
TTL_RECORDS	optional	Layer 2 only — u16 n_ttl · (rule_idx:u16 · ttl_sessions:u8)*

Table 2. v0.4.0 wire format section layout.

The FLAGS field is a u16 bitmask:

Flag	Hex	Meaning
FLAG_DELTA_ENCODED	0x0001	Support values stored as zigzag delta from previous rule (sorted descending)
FLAG_HAS_TTL	0x0002	TTL_RECORDS section present at end of blob (Layer 2 only)
FLAG_SESSION_RLE	0x0004	SESSION_RUNS section present; v0.4 blobs always set this flag — NEW

Table 3. Wire format flag definitions.

3.2 Change A — Variable-Length Integer Indices

In v0.3, lhs_idx and rhs_idx were encoded as fixed u16 (2 bytes each, 4 bytes per rule for both). In v0.4, both are encoded as unsigned LEB128 varints.

LEB128 encoding uses 7 bits of payload per byte; the high bit is a continuation flag. Values 0–127 encode in 1 byte. Values 128–16,383 encode in 2 bytes. For typical FBG vocabulary (≤ 127 unique tokens per behavioral domain), every index encodes in 1 byte.

Vocab size	v0.3 bytes/index	v0.4 bytes/index	Saving per rule (2 indices)
≤ 127 tokens (typical)	2	1	2 bytes
128–16,383 tokens	2	2	0 bytes (break-even)
$> 16,383$ tokens	2	3	-2 bytes (larger)

Table 4. VLQ index savings by vocabulary size.

Applied to the 117-rule test corpus (vocab = 23 unique strings): saving = $1\text{B} \times 2 \text{ indices} \times 117 \text{ rules} = 234$ bytes. This is pure saving with zero added algorithmic complexity — varint primitive routines were already present in the codebase for support delta encoding.

3.3 Change B — Run-Length Encoded Session Count

In v0.3, each rule record carried a packed byte containing layer_id (2 bits) + session_count (6 bits). In v0.4, session_count is extracted from the per-rule payload and stored in a dedicated SESSION_RUNS section as run-length encoded pairs.

The SESSION_RUNS section precedes RULE_RECORDS and encodes (count, value) pairs: consecutive rules with the same session_count emit one pair. The layer_id bits are removed from the per-rule packed byte entirely — layer_id is already present in the 8-byte header and does not need to be repeated per rule.

Per-rule payload change:

Field	v0.3 per-rule	v0.4 per-rule	Change
lhs_idx	u16 (2B)	varint (1B typical)	-1B (Change A)

rhs_idx	u16 (2B)	varint (1B typical)	-1B (Change A)
support	varint zigzag-delta	varint zigzag-delta	unchanged
conf_q8	u8 (1B)	u8 (1B)	unchanged
packed byte	u8 (1B)	removed	-1B (Change B)
session_count	(in packed byte)	(in SESSION_RUNS)	moved to section header

Table 5. Per-rule payload comparison v0.3 → v0.4.

The SESSION_RUNS section overhead is 2 bytes (u16 n_runs) + 2 bytes per run (count:u8 + value:u8). Break-even for Layer 2 (all session_count = 1) is $n = 1$ rule. Break-even for Layer 1 (session_count in {2,3,4}) is approximately $n = 4$ rules. Above those thresholds, RLE is always beneficial.

3.4 Change C — Why RLE on conf_q8 Was Rejected

Change C was evaluated and explicitly rejected. The argument for RLE on conf_q8 (quantized confidence, u8) was that confidence values might cluster and compress well under RLE.

The problem is sort order conflict. Rules are sorted descending by support to enable zigzag delta encoding on the support field — that delta encoding saves bytes only when support values are sorted. The same sorted order randomizes the conf_q8 sequence, because confidence is not correlated with support. RLE on an unsorted field produces run lengths averaging approximately 1.0, meaning every run is a singleton pair — overhead with no saving.

Change	Complexity	Saving (117 rules)	Decision
A — VLQ indices	Zero (primitives existed)	+234 bytes (19%)	Adopted
B — RLE session_count	Low (new section header)	+99 bytes (8%)	Adopted
C — RLE conf_q8	Medium-high	0 to negative	Rejected

Table 6. Encoding change decision matrix.

4. Storage & Latency Benchmarks

4.1 Storage Comparison

Architecture	Rules	Storage	vs. flat v0.2.0	vs. JSON
Flat v0.2.0 (JSON, 20-rule hard cap)	20	2,104 B	1.0× (baseline)	1.0×
Overdub v0.3.0 (JSON serialization)	117	25,012 B	11.9× larger	1.0×
Overdub v0.3.0 (binary, v0.3 format)	117	1,203 B	0.57×	20.8×
Overdub v0.4.0 (binary, VLQ + RLE)	117	860 B	0.41×	29.1×

Overdub v0.4.0 vs. JSON (same 117 rules)	117	860 B	—	10.9×
--	-----	-------	---	-------

Table 7. Storage comparison across architecture versions. Overdub v0.4.0 stores 5.75× more validated rules than the flat cap at 41% of the original storage cost.

The v0.4.0 per-layer breakdown for the 117-rule test corpus:

Layer	Rules	v0.3 blob	v0.4 blob	Saving
Layer 0 — Domain	25	351 B	280 B	-71 B (-20%)
Layer 1 — Behavioral	92	852 B	580 B	-272 B (-32%)
Combined	117	1,203 B	860 B	-343 B (-28%)

Table 8. Per-layer byte savings v0.3 → v0.4.

4.2 Latency Benchmark (fg_gate_rs v0.4.0)

All measurements taken on the maturin release build (Rust 1.70+, PyO3 0.21). Python reference encoder implemented identically for comparison.

Operation	Rust v0.4.0	Python ref	Speedup
Encode L0 (25 rules)	15.3 μs	—	—
Encode L1 (92 rules)	48.7 μs	~194 μs	~4.0×
Decode L0	14.8 μs	—	—
Decode L1	50.0 μs	—	—
Gate rebuild (200 entries, 10 sessions)	152 μs	—	startup only
Encode L0 + L1 combined	64 μs	~194 μs	3.0×

Table 9. Latency benchmark results. The 64 μs combined encode/decode target applies to session-boundary persistence — the operation that fires at every session end. Gate rebuild (152 μs) fires once at startup and is not on the inference hot path.

5. Backward Compatibility

The decoder detects the version field in the 8-byte HEADER and dispatches to the appropriate decode path. v0.3 blobs (VERSION = 0x03) are decoded correctly by the v0.4 decoder using a dedicated backward-compat path that:

- reads per-rule packed bytes (layer_id + session_count bits present)
- treats lhs_idx and rhs_idx as fixed u16 (no VLQ decode)
- expects no SESSION_RUNS section (FLAG_SESSION_RLE = 0x00)

v0.4 blobs set FLAG_SESSION_RLE = 0x0004 in the header flags field. The encoder always writes VERSION = 0x04 for new blobs. There is no upgrade path — existing v0.3 blobs are read and decoded correctly, and the next write for that layer will produce a v0.4 blob.

The 40 Rust unit tests include explicit backward-compat coverage:

`test_v3_backward_compat_decode` constructs a valid v0.3 blob and verifies that the v0.4 decoder produces identical rule data. All 40 Rust tests and 15 Python integration tests pass with zero warnings.

6. Implementation Reference

6.1 Repository Structure

Source: github.com/GreenbarSystems/fractal-grammar

<code>fg_gate_rs/src/format.rs</code>	Wire format constants, BinaryRule struct, varint helpers, RLE helpers
<code>fg_gate_rs/src/encode.rs</code>	Batch encoder — VLQ indices, SESSION_RUNS section
<code>fg_gate_rs/src/decode.rs</code>	Batch decoder — VLQ decode, RLE session_count, v0.3 backward compat
<code>fg_gate_rs/src/gate.rs</code>	EphemeralGate — rebuilds from event log, 8-bit circular bitmask
<code>fg_gate_rs/src/tests.rs</code>	40 Rust unit tests (was 31; 9 new v0.4 tests added)
<code>fg_gate_rs/tests/ test_integration.py</code>	15 Python integration tests + latency benchmark (was 13)
<code>fractal_grammar/pipeline.py</code>	5-stage extraction pipeline (unchanged from v0.1.0)
<code>fg_sync/server.py</code>	HTTP proxy: port 11435 → Ollama 11434

6.2 Quick Integration

Install the Rust binary extension (requires Rust 1.70+ and maturin):

```
git clone https://github.com/GreenbarSystems/fractal-grammar
cd fractal-grammar/fg_gate_rs
pip install maturin
./build.sh
```

Encode and decode a grammar layer:

```
from fg_gate import encode_rules, decode_rules, promote_gate
from fg_gate import LAYER_DOMAIN, LAYER_BEHAVIORAL

# Encode extracted rules into binary blob
blob = encode_rules(rules, layer_id=LAYER_BEHAVIORAL) # → bytes
decoded_rules, layer_id = decode_rules(blob)

# Rebuild ephemeral promotion gate from raw event log
# events = [(session_index: int, lhs: str, rhs: str), ...]
result = promote_gate(events, l0_threshold=5, l1_threshold=2)
# result['layer0'], result['layer1'], result['layer2'], result['stats']
```

7. Prior Art Disclosure

This repository and whitepaper constitute a public prior art disclosure as of June 2026. The specific novel claim covered by provisional patent FBG-2026-001 is:

CORE CLAIM

Using Hurst exponent self-similarity detection across HDBSCAN hierarchical cluster resolution levels, encoded via hyperdimensional computing and compressed through a three-layer hierarchical residual architecture with binary-encoded rule persistence (VLQ indices, RLE session metadata), as the criterion for identifying and persisting compressible behavioral grammar rules from LLM interaction logs, motivated by the neuroscientific distinction between episodic memory (CA1 hippocampus) and rule-based memory (medial prefrontal cortex), for the purpose of compression-first local AI personalization.

Component attribution:

Component	Prior Art	Novel Application Here
Hurst exponent / R/S analysis	Mandelbrot & Wallis (1969)	First use as compression criterion for LLM behavioral logs
HDBSCAN clustering	Campello et al. (2013)	First cross-level application with self-similarity tracing
MinHash LSH novelty gating	Broder (1997)	First use as behavioral novelty gate controlling grammar ingestion
Hyperdimensional computing (HDC)	Kanerva (2009)	First use as behavioral embedding layer in grammar extraction
Hierarchical residual compression	Video codec P/B-frame design	First application to behavioral grammar persistence
mPFC rule / CA1 episode distinction	Neuroscience, 1990s–present	First use as architectural design rationale for grammar / episode separation

Table 10. Novel composition components and prior art attribution.

Patent docket FBG-2026-001 was filed as a provisional application in June 2026. The non-provisional deadline is June 29, 2027. The provisional filing date establishes the priority date for all claims disclosed herein.

Recommended citation:

Moore, R. (2026). Fractal Behavioral Grammar: Overdub Architecture and Binary Encoding (v0.3.0–v0.4.0). Greenbars Systems. GitHub: github.com/GreenbarSystems/fractal-grammar. Prior art disclosure, June 2026.

This document was generated by Perplexity Computer on behalf of Ryan Moore, Greenbars Systems, Goodyear, Arizona. Apache 2.0 license. Repository: github.com/GreenbarSystems/fractal-grammar.